

FUNCTIONAL SAFETY

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Airbag Electronic Control Unit (ECU)

ISO 26262:2018 / IEC 61508:2010 | ASIL D / SIL 3

1. Document Information

Document Title	Software Architecture and Design Specification
Document Number	SADS-AIRBAG-ECU-001
Revision	1.0
Date	2026-05-31
System	Airbag Electronic Control Unit (ECU)
Standards	ISO 26262:2018 / IEC 61508:2010 / ISO 11898
ASIL / SIL	ASIL D / SIL 3
Classification	Confidential Functional Safety Document
Based On	SRS-AIRBAG-SW-001 Rev 1.0 FSRs-AIRBAG-ECU-001 Rev 2.1
Author	Software Safety Engineering Team

1.1 Revision History

Rev	Date	Author	Description
1.0	2026-05-28	Software Safety Engineering Team	Initial release

2. Purpose and Scope

This Software Architecture and Design Specification (SADS) describes how the Airbag ECU software is organized internally: module decomposition, data flows, control flows, real-time scheduling, safety mechanisms, state machine, and pseudo-code of key algorithms.

It is produced in accordance with:

- ISO 26262:2018 Part 6 - Product Development: Software Level
- IEC 61508:2010 Part 3 - Software Requirements
- ISO 11898 - Road Vehicles: Controller Area Network (CAN)
- FSRs-AIRBAG-ECU-001 Rev 2.1 - Functional Safety Requirements Specification

The document covers embedded software on both Arduino Due channels (Channel A, Channel B). Hardware design, inflators and vehicle body structure are outside scope.

3. System Context and Architecture Overview

3.1 Dual-Channel Redundant Architecture

The ECU uses a two-out-of-two (2oo2) dual-channel redundant architecture. Two independent Arduino Due microcontroller boards (Channel A – Channel B) execute identical software, independently read all sensor inputs, independently calculate deployment decisions, and communicate their status via UART serial interface. A deployment command is authorized only when both channels agree on a valid crash event with no active faults.

The physical 2oo2 deployment constraint is enforced by a hardware DM74LS08N AND gate: both boards must simultaneously assert their D8 line HIGH for the deployment signal to propagate. Software alone cannot trigger deployment.

A separate setup CAN Sensor Transmitter reads the same three potentiometer-based sensor inputs and transmits raw ADC values to the OMS laboratory PC via CAN bus at 500 kbps. This channel is classified as ASIL B / SIL 2 as it carries diagnostic/monitoring data, not deployment decisions.

Layer	Component	Function	ASIL	Interface
Sensor Layer	3× potentiometers (A0/A1/A2) 1× digital switch D11	Simulate crash sensor signals and baby-seat occupancy	-	Analog 0–3.3 V / Digital GPIO
Processing - Due 1 (Channel A, Master)	Arduino Due SAM3X8E 84 MHz ARM Cortex-M3	Sensor acquisition, crash detection, deploy decision, CAN TX to OMS PC	ASIL D	GPIO D8/D9/D10/D11 UART Serial1 ↔ Due 2 CAN0 → Waveshare
Processing - Due 2 (Channel B, Slave)	Arduino Due SAM3X8E 84 MHz ARM Cortex-M3	Sensor acquisition, crash detection, deploy decision, UART status to Due 1	ASIL D	GPIO D8/D9/D10/D11 UART Serial1 ↔ Due 1
2oo2 Gate	DM74LS08N AND gate	Physical enforcement of dual-channel agreement — both D8 lines required HIGH	ASIL D	Hardware gate — no software bypass possible
Communication	Waveshare SN65HVD230 CAN transceiver	CAN bus bridge: Due 1 CAN0 → USB-CAN-A → OMS laboratory PC	ASIL B	CAN bus 500 kbps / ISO 11898

This architecture satisfies:

- ISO 26262 ASIL D - Single-Point Fault Metric (SPFM) targets
- IEC 61508 SIL 3 - Probability of Failure on Demand
- SC-03: No single-point fault shall cause unintended deployment
- SC-04: Deployment requires agreement of both redundant channels

3.2 Hardware Platform

Both processing channels execute on Arduino Due boards featuring the Atmel SAM3X8E ARM Cortex-M3 microcontroller. Key hardware resources utilized:

- 12-bit ADC - three analog sensor channels (A0, A1, A2)
- Digital GPIO - baby-seat detection (D11), airbag outputs (D8, D9), warning lamp (D10), status LED (D13)
- Hardware Watchdog Timer (WDT) - runaway software detection
- UART (Serial/Serial1) - inter-ECU heartbeat communication

- 115200 baud serial interface - monitoring and diagnostics
- CAN bus at 500 kbp: module (transceiver) and USB-CAN adapter - connect CAN bus to OMS lab PC

Board	Role	CAN	UART
Arduino Due 1 (Channel A - Master)	Reads sensors, drives AND gate inputs, sends CAN frames to OMS PC, parses Due 2 status	CAN0 TX → Waveshare SN65HVD230 → USB-CAN-A → OMS PC	TX1/RX1 ↔ Due 2
Arduino Due 2 (Channel B - Slave)	Reads sensors, drives AND gate inputs, sends status string to Due 1	None - no CAN module	TX1/RX1 ↔ Due 1

Due 1 is Master and the only board with CAN capability (via Waveshare SN65HVD230 transceiver). Due 2 is Slave with no CAN. CAN is not a separate physical board — it is Due 1's communication interface to the OMS PC.

3.3 Safety-Relevant vs. Non-Safety Software

Safety-Relevant Functions (ASIL C–D)	Non-Safety / Diagnostic Functions (ASIL A–B)
POST self-tests: cpu_test, ram_test, adc_test Watchdog setup and feed Sensor read + toPhys conversion Threshold comparison + OR fusion Deployment drive logic (D8/D9) Fault aggregation + safe-state enforcement UART heartbeat TX/RX + timeout detection Due 2 status parsing	DWI lamp (D10) - ASIL A CAN communication to OMS PC - ASIL B Serial monitor diagnostic print - ASIL A Rolling sequence counter in CAN frame

4. Software Constants and Thresholds

All safety-critical constants are defined as compile-time literals (not variables) and are traceable to their governing FSR. These values are used across multiple modules and must not be modified without a change impact analysis.

Constant	Value	Unit	Meaning	FSR
ADC_MAX	4095.0	-	Arduino Due 12-bit ADC maximum	FSR-01
ACCEL_MAX	100.0	g	Accelerometer full-scale range	FSR-01
GYRO_MAX	360.0	deg/s	Gyroscope full-scale range	FSR-01
PRESS_MAX	500.0	kPa	Pressure sensor full-scale range	FSR-01
THRESH_ACCEL	30.0	g	Frontal crash detection threshold	FSR-03
THRESH_GYRO	120.0	deg/s	Rollover detection threshold	FSR-03
THRESH_PRESS	200.0	kPa	Side impact detection threshold	FSR-06
DEBOUNCE_MS	10	ms	Baby seat pin debounce time	FSR-11
HEARTBEAT_MS	500	ms	UART heartbeat transmit interval	FSR-07
COMM_TIMEOUT	2000	ms	UART link fault threshold	FSR-15

CAN_TX_MS	100	ms	CAN frame transmit interval	FSR-16
CAN_TX_ID	0x100	hex	CAN message identifier for OMS PC	FSR-16
CAN_BAUD	CAN_BPS_500K	-	CAN bus speed 500 kbit/s	FSR-13

5. Software Architecture

5.1 Architectural Pattern

Both boards use a cyclic executive (super-loop) with no RTOS. All modules execute in a fixed, sequential order every cycle. Target cycle time < 1 ms (SWR-NF-01). All timing uses non-blocking millis() comparisons (SWR-11). No delay() is permitted in the main loop.

5.2 Programming Language Requirements

For prototyping purposes, the software for both channels of the 2oo2 Airbag ECU is developed within the **Arduino Ecosystem (Embedded C)** targeting the Atmel SAM3X8E microcontrollers. Arduino IDE 2.3.6 is used.

While unconstrained Arduino frameworks are typically considered inappropriate for commercial SIL 3 systems due to hidden background abstractions, this architecture introduces a strict, customized coding subset to bridge the gap between academic hardware and safety-critical compliance (IEC 61508-3 Table A.4).

5.3 Module Decomposition

The table below maps every function and code block from SRS-AIRBAG-SW-001 to a module identifier used throughout this document. 'Runs On' reflects the board assignment from the SRS.

ID	Function / Block	Runs On	Cycle	ASIL	SWR	Key Responsibility
MOD-01	cpu_test()	Both	Boot	ASIL C	SWR-01	Arithmetic self-test - 12345×6789=83 810 205
MOD-02	ram_test()	Both	Boot	ASIL D	SWR-01	Checkerboard write/read pattern - 0xAAAA / 0x5555
MOD-03	adc_test()	Both	Boot	ASIL C	SWR-01	Verify analogRead(A0) ∈ [0, 4095]
MOD-04	watchdog_setup()	Both	Boot	ASIL D	SWR-15	Configure SAM3X8E WDT, 10 ms timeout, reset-on-trip
MOD-05	watchdog_feed()	Both	< 10 ms	ASIL D	SWR-16	First call in every loop(); kicks WDT
MOD-06	updateBabySeat()	Both	Every loop	ASIL D	SWR-07–09	Read D11, 10 ms debounce, suppress-on-fault
MOD-07	toPhys()	Both	Every loop	ASIL D	SWR-05	Scale raw ADC → physical unit: (raw/4095.0)×MAX
MOD-08	Sensor read block	Both	Every loop	ASIL D	SWR-04	analogRead(A0/A1/A2) → toPhys() → physical values

MOD-09	Threshold check block	Both	Every loop	ASIL D	SWR-06	Compare physical values → accelEx/gyroEx/pressEx flags
MOD-10	crashLocal decision	Both	Every loop	ASIL D	SWR-10	OR-fusion: crashLocal = accelEx gyroEx pressEx
MOD-11	driveAirbag1/2 logic	Both	Every loop	ASIL D	SWR-12–14	Evaluate 3-condition gate; drive D8/D9 HIGH or LOW
MOD-12	DWI logic	Both	Every loop	ASIL A/D	SWR-24–25	Set D10 HIGH on any local or remote fault
MOD-13	enterSafeState logic	Both	Every loop	ASIL D	SWR-14	Force D8/D9 LOW when system_fault TRUE
MOD-14	UART heartbeat TX	Both	500 ms	ASIL D	SWR-17	Non-blocking millis() TX - 'ECU1:HB' or 'ECU2:HB'
MOD-15	UART heartbeat RX	Both	Every loop	ASIL D	SWR-18	Parse Serial1 RX; reset timeout timer on valid HB
MOD-16	UART timeout check	Both	Every loop	ASIL D	SWR-18	Set fault_due1/2 if no HB in 2000 ms
MOD-17	parseUART() / status TX	Due 1 + Due 2	100 ms	ASIL D	SWR-19	Due 2 TX: 'ECU2,<c>,,<f>\n'; Due 1 parses it
MOD-18	crc16()	Due 1	On CAN TX	ASIL B	SWR-22	CRC-16 poly 0x1021, seed 0xFFFF over 6 bytes
MOD-19	canSend()	Due 1	100 ms	ASIL B	SWR-20–23	Pack 8-byte frame, append CRC-16, TX via Can0
MOD-20	Serial monitor output	Both	1000 ms	ASIL A	-	Diagnostic print - fault/sensor/crash/deploy summary

5.4 Module Descriptions

5.4.1 POST Modules - MOD-01 to MOD-04 (Boot, Both Boards)

Executed once before loop() is entered. cpu_test() verifies arithmetic (12345×6789=83 810 205). ram_test() writes 0xAAAA/0x5555 patterns across a 128-word buffer and verifies them. adc_test() reads A0 and confirms the result is in [0,4095]. watchdog_setup() configures the SAM3X8E WDT with WDT_MR_WDRSTEN and WDT_MR_WDV(256×10) for a ~10 ms timeout. No airbag output is set HIGH until all POST results are evaluated (SWR-02).

5.4.2 Watchdog Module - MOD-05 (Every Loop, Both Boards)

watchdog_feed() is the unconditional first call in loop(). Writes service key 0xA5 to WDT_CR. If the loop stalls for > 10 ms the WDT resets the MCU, driving all GPIO to their default LOW state before re-initialization.

5.4.3 Baby Seat - MOD-06 (Every Loop, Both Boards)

updateBabySeat() reads D11 (configured as INPUT — external 10 kΩ pull-up to 3.3 V per SWR-07). Active-LOW: LOW = baby seat present. A state change is accepted only after 10 ms of stable signal (SWR-08). If the signal transitions faster than 10 ms or is absent, babySeat remains true (suppress-on-fault, SWR-09).

5.4.4 Sensor Processing - MOD-07 to MOD-10 (Every Loop, Both Boards)

toPhys() converts raw ADC counts: $\text{physical} = (\text{raw} / 4095.0) \times \text{MAX_VALUE}$ (SWR-05). The sensor read block calls analogRead(A0/A1/A2) and passes results to toPhys() (SWR-04). The threshold check block compares physical values against THRESH_ACCEL/GYRO/PRESS and sets per-sensor boolean flags (SWR-06). crashLocal is set TRUE by OR-fusion of the three flags (SWR-10).

5.4.5 Deployment & Fault Logic - MOD-11 to MOD-13 (Every Loop, Both Boards)

driveAirbag1 logic: D8 = HIGH if crashLocal AND NOT system_fault AND NOT fault_dueX (SWR-12). driveAirbag2 logic: D9 = HIGH additionally if NOT babySeat (SWR-13). enterSafeState: if system_fault TRUE, D8 and D9 are immediately driven LOW (SWR-14). DWI logic: D10 = HIGH on any local fault (fault_cpu/ram/adc/due1/due2) or on received due2_fault flag (SWR-24, SWR-25).

5.4.6 UART Communication - MOD-14 to MOD-17 (Both Boards / Due 2 → Due 1)

Both boards transmit 'ECU1:HB' or 'ECU2:HB' every 500 ms via Serial1 at 115200 baud (SWR-17). Each monitors for the peer heartbeat; if none received in 2000 ms, fault_due1 or fault_due2 is set (SWR-18). Additionally, Due 2 sends 'ECU2,<crash>,<baby>,<fault>' every 100 ms; Due 1 parses this into due2_crash, due2_baby, due2_fault (SWR-19).

5.4.7 CAN Communication - MOD-18 to MOD-19 (Due 1 Only)

crc16() computes CRC-16 with polynomial 0x1021 and seed 0xFFFF over the first 6 bytes of the CAN frame (SWR-22). canSend() builds an 8-byte frame (ID 0x100, DLC=8), appends the CRC-16 in bytes 6–7 (big-endian), increments the rolling sequence counter in byte 5, and calls Can0.sendFrame() every 100 ms (SWR-20, SWR-21, SWR-23). Due 1 only: Due 2 has no CAN module.

6. Data Flow

The data flow table is divided into four layers separated by labelled divider rows. Data flows strictly from sensor inputs through processing modules to actuator outputs. No feedback loops exist in the deployment decision path. The CAN communication layer is independent and carries monitoring data only.

ID	Data Item	Producer	Consumer	Type	Rate	Feeds
Sensor Processing Layer - Both Boards (MOD-07 to MOD-10)						
DF-01	Raw ADC (A0,A1,A2)	HW ADC	SensorAcq (MOD-08)	int[3]	Every loop	DF-02
DF-02	Physical values (g / °s / kPa)	toPhys (MOD-07)	ThresholdCheck (MOD-09)	float[3]	Every loop	DF-03
DF-03	Threshold flags (accelEx/gyroEx/pressEx)	ThresholdCheck (MOD-09)	crashLocal (MOD-10)	bool[3]	Every loop	DF-04
DF-04	crashLocal flag	crashLocal (MOD-10)	Deploy (MOD-11)	bool	Every loop	DF-07
DF-05	Baby-seat raw (D11)	GPIO D11	updateBabySeat (MOD-06)	bool	Every loop	DF-06
DF-06	babySeat (debounced)	updateBabySeat (MOD-06)	Deploy (MOD-11)	bool	Every loop	DF-07
Deployment & Diagnostics Layer - Both Boards (MOD-11 to MOD-13)						
DF-07	system_fault (OR of all faults)	Fault aggregation	Deploy (MOD-11) DWI (MOD-12) SafeState (MOD-13)	bool	Every loop	DF-08/09/10

DF-08	D8 drive command (Airbag 1)	Deploy (MOD-11)	GPIO D8 → AND gate	bool	Every loop	-
DF-09	D9 drive command (Airbag 2)	Deploy (MOD-11)	GPIO D9 → AND gate	bool	Every loop	-
DF-10	D10 drive command (DWI lamp)	DWI (MOD-12)	GPIO D10	bool	Every loop	-
UART Inter-Board Communication - Both Boards (MOD-14 to MOD-17)						
DF-11	'ECU1:HB' / 'ECU2:HB' message	UART HB TX (MOD-14)	UART TX → Serial1	String	500 ms	-
DF-12	Peer heartbeat RX	Serial1 RX	UART HB RX (MOD-15)	String	500 ms	DF-13
DF-13	fault_due1 / fault_due2 flag	UART timeout (MOD-16)	Fault aggregation	bool	Every loop	DF-07
DF-14	'ECU2,<c>,,<f>' status string	Due 2 status TX (MOD-17)	Due 1 parseUART (MOD-17)	String	100 ms	DF-15
DF-15	due2_crash, due2_baby, due2_fault	parseUART (MOD-17)	Deploy / DWI / Fault aggr.	bool[3]	100 ms	DF-07/10
CAN Communication Interface - Due 1 Only (MOD-18 to MOD-19)						
DF-16	8-byte CAN payload (flags+sensors+CRC)	canSend (MOD-19)	CAN Bus → OMS PC	CAN_FRAME	100 ms	-
DF-17	CRC-16 (poly 0x1021, seed 0xFFFF)	crc16 (MOD-18)	canSend (MOD-19) — bytes 6–7	uint16	On TX	DF-16

6.1 Critical Deployment Data Flow Path

From sensor threshold exceedance to GPIO output. Must complete within < 1 ms per loop cycle:

- HW ADC → MOD-08: raw readings sampled (DF-01)
- MOD-08 → MOD-07 → MOD-09: toPhys conversion → threshold flags (DF-02 → DF-03)
- MOD-09 → MOD-10: OR-fusion → crashLocal (DF-03 → DF-04)
- D11 → MOD-06: debounce → babySeat (DF-05 → DF-06)
- Fault aggregation → MOD-11/13: system_fault gates D8/D9 (DF-07 → DF-08/09)

6.2 UART Cross-Board Path

Two independent UART data paths run concurrently:

- Heartbeat path (DF-11 → DF-12 → DF-13): 500 ms HB TX, 2000 ms timeout → fault_dueX
- Status path (DF-14 → DF-15): Due 2 sends crash/baby/fault every 100 ms → Due 1 parses and uses in DWI and remote fault logic

6.3 CAN Communication Path (Due 1 Only)

Independent from the deployment path:

- Deployment status + sensor data → MOD-18: crc16() over bytes 0–5 (DF-17)
- MOD-18 → MOD-19: canSend() builds 8-byte frame with CRC + sequence counter → CAN bus → OMS PC (DF-16)

A CAN TX failure does not affect `system_fault` and cannot influence the deployment decision, it operates independently of CAN at all times. CAN is ASIL B; deployment logic is ASIL D.

7. Control Flow and Real-Time Scheduling

7.1 Execution Sequence

The software executes in two phases: startup initialization (single execution) and the main cyclic loop (continuous execution). The sequence within each phase is strictly ordered.

7.1.1 Startup Sequence - Both Boards (`setup()`)

1. `Serial.begin(115200)` - diagnostic monitoring
2. `Serial1.begin(115200)` - UART inter-board link
3. `analogReadResolution(12)` - 12-bit ADC mode (SWR-03)
4. `pinMode`: D11 INPUT, D8/D9/D10 OUTPUT; D8/D9/D10 driven LOW
5. Due 1 only: `Can0.begin(CAN_BPS_500K, 255)` - CAN0 at 500 kbps (SWR-20)
6. `watchdog_setup()` - configure WDT ~10 ms timeout (SWR-15)
7. `cpu_test()` → `fault_cpu`; `ram_test()` → `fault_ram`; `adc_test()` → `fault_adc` (SWR-01)
8. Evaluate POST results; no output arming before this point (SWR-02)
9. Initialise timing variables: `last_hb_tx`, `last_can_tx`, `last_status_tx`, `last_print`
10. Enter `loop()`

7.1.2 Main Loop Execution - Both Boards (`loop()`, `target < 1 ms`)

11. Strictly ordered. All timing via `millis()`. No blocking `delay()`:
12. `watchdog_feed()` — unconditionally first (SWR-16)
13. `updateBabySeat()` — read D11, apply 10 ms debounce (SWR-07/08/09)
14. UART HB TX — if `millis()` - `last_hb_tx` ≥ 500 ms: transmit 'ECUn:HB', reset timer (SWR-17)
15. UART HB RX — read `Serial1`; if valid HB found: reset timeout timer, clear `fault_dueX` (SWR-18)
16. UART timeout check — if `millis()` - `last_rx` > 2000 ms: `fault_dueX` = true (SWR-18)
17. Due 2 only: if `millis()` - `last_status_tx` ≥ 100 ms: TX 'ECU2,<c>,,<f>' (SWR-19)
18. Due 1 only: parse incoming `Serial1` for ECU2 status string → `due2_crash/baby/fault` (SWR-19)
19. `analogRead(A0/A1/A2)` → `toPhys()` → `accel_g`, `gyro_dps`, `press_kPa` (SWR-04/05)
20. Threshold comparisons → `accelEx`, `gyroEx`, `pressEx` (SWR-06)
21. `crashLocal` = `accelEx` || `gyroEx` || `pressEx` (SWR-10)
22. Fault aggregation: `system_fault` = `fault_cpu` || `fault_ram` || `fault_adc` || `fault_due1` || `fault_due2`
23. `enterSafeState`: if `system_fault` → D8=LOW, D9=LOW (SWR-14)
24. `driveAirbag1`: D8 = `crashLocal` && !`system_fault` (SWR-12)
25. `driveAirbag2`: D9 = `crashLocal` && !`babySeat` && !`system_fault` (SWR-13)
26. DWI: D10 = `system_fault` || `due2_fault` (SWR-24/25)
27. Due 1 only: if `millis()` - `last_can_tx` ≥ 100 ms: `crc16()` → `canSend()` (SWR-22/23)
28. Serial print if `millis()` - `last_print` ≥ 1000 ms
29. Return to 1

All timing constraint values (< 1 ms loop, 500 ms HB, 2000 ms timeout, 100 ms CAN) are defined in SRS-AIRBAG-SW-001 §6 Non-Functional Requirements.

The execution sequence, including the initialization phase, heartbeat communication, sensor fusion thresholds, and the multi-fault 'Safe State' routing, is modeled in a dedicated high-resolution technical

drawing. Due to the scale and level of detail required for verification, the visual representation is maintained as an external artifact. Please refer to [AD-01] (2oo2 Airbag ECU - Detailed Software Flowchart) attached to this specification package for the complete control flow visualization.

8. Software Safety Mechanisms

The following table enumerates all software safety mechanisms implemented in the ECU. Each mechanism is traceable to a specific safety requirement and ASIL classification.

ID	Mechanism	Layer	ASIL	Description
SM-01	Watchdog (WDT)	Hardware	ASIL D	10 ms timeout; watchdog_feed() first call every loop; reset on expiry → MCU restart (SWR-15, SWR-16)
SM-02	CPU arithmetic self-test	POST (MOD-01)	ASIL C	$12345 \times 6789 = 83\,810\,205$ verified; fault_cpu set on fail (SWR-01)
SM-03	RAM checkerboard test	POST (MOD-02)	ASIL D	0xAAAA / 0x5555 write/verify patterns; fault_ram set on fail (SWR-01)
SM-04	ADC range plausibility	POST (MOD-03)	ASIL C	$\text{analogRead}(A0) \in [0, 4095]$; fault_adc set on fail (SWR-01)
SM-05	2oo2 hardware AND gate	Hardware	ASIL D	DM74LS08N AND gate: both D8 lines must be HIGH simultaneously — physical enforcement of dual-channel agreement
SM-06	Baby-seat 10 ms debounce	MOD-06	ASIL D	State change only accepted after 10 ms stable signal (SWR-08)
SM-07	Suppress-on-fault (baby seat)	MOD-06	ASIL D	Default babySeat=true if signal intermittent or absent (SWR-09)
SM-08	Deploy inhibit on system_fault	MOD-11/13	ASIL D	D8 and D9 driven LOW if any fault flag active (SWR-14)
SM-09	UART heartbeat timeout	MOD-16	ASIL D	fault_due1/2 = true if no HB within 2000 ms; inhibits deployment (SWR-18)
SM-10	Due 2 status cross-check	MOD-17	ASIL D	Due 1 receives crash/baby/fault from Due 2 every 100 ms; used in DWI and remote fault logic (SWR-19)
SM-11	CAN CRC-16 protection	MOD-18	ASIL B	Poly 0x1021, seed 0xFFFF over bytes 0–5; receiver validates integrity (SWR-22)
SM-12	CAN rolling sequence counter	MOD-19	ASIL B	Byte 5 increments every TX; OMS PC detects missed frames (SWR-21)
SM-13	No blocking code in main loop	Architecture	ASIL D	All timing via millis(); no delay(); loop target < 1 ms (SWR-11, SWR-NF-01)

8.1 Hardware AND Gate - 2oo2 Physical Enforcement (SM-05)

The DM74LS08N AND gate is the physical implementation of the 2oo2 requirement. Due 1 drives one AND gate input; Due 2 drives the other. The output is HIGH (deployment signal) only when both inputs are simultaneously HIGH. This cannot be bypassed by software on either board alone.

8.2 Watchdog - 10 ms Timeout (SM-01)

The SAM3X8E WDT is configured with WDT_MR_WDRSTEN (reset-on-timeout) and WDT_MR_WDV(256×10). watchdog_feed() is the first instruction executed every loop iteration. A stall longer than ~10 ms triggers a hardware reset, returning all GPIO to their default LOW state.

8.3 Baby-Seat Suppress-on-Fault

If the baby-seat signal becomes unstable, intermittent, or unavailable, the module defaults to babySeatPresent = true, which inhibits Airbag 2 (passenger airbag). This ensures that an uncertain or degraded occupancy signal always results in the safer outcome of suppressing rather than deploying the passenger airbag.

8.4 Deploy Inhibit on System Fault

The Deploy_Module evaluates the composite system_fault flag as a gate condition before any deployment output is activated. The deployment authorization logic is:

```
deployAirbag1 = crashDetected AND NOT system_fault
deployAirbag2 = crashDetected AND NOT babySeatPresent AND NOT system_fault
```

This ensures that any active fault, regardless of source, prevents airbag deployment. This is a deliberate design choice: the system prioritizes prevention of unintended deployment (Safety Goal SG-01, ASIL D) over deployment in potentially uncertain conditions.

8.5 UART Heartbeat + Status Cross-Check (SM-09, SM-10)

Two concurrent UART supervision mechanisms run simultaneously. The heartbeat (500 ms, 2000 ms timeout) detects total loss of communication. The Due 2 status string (100 ms) provides fine-grained remote state: crash, baby seat, and fault flags are received by Due 1 and factored into the DWI and remote fault logic. Neither mechanism can be spoofed without physical access to the UART bus.

8.6 CAN Frame Integrity - CRC-16 and Sequence Counter (SM-11, SM-12)

CRC-16 (polynomial 0x1021, seed 0xFFFF) is computed over the first 6 bytes of every CAN frame and appended in bytes 6–7 (big-endian). The OMS PC validates the CRC on every received frame. The rolling sequence counter in byte 5 (wrapping 0–255) allows the OMS PC to detect any missed frames even if CRC passes.

9. State Machine and Deployment Logic

9.1 ECU System State Machine

The state machine is implicit, driven by the values of fault flags, crashLocal, babySeat, and the AND gate hardware state. Both boards execute this logic independently in every loop iteration.

State	Entry Condition	Key Flags	Behaviour
IDLE	POST complete; no crash; no fault	system_fault=0 crashLocal=0	D8=LOW, D9=LOW, D10=LOW; WDT fed; UART monitored
MONITORING	Normal operation, sensor polling	system_fault=0 all outputs LOW	Sensors read every loop; heartbeats TX every 500 ms; CAN TX every 100 ms (Due 1)

CRASH DETECTED	crashLocal=true on any board	crashLocal=1 system_fault=0	D8=HIGH (driver airbag armed); D9=HIGH if babySeat=false; AND gate passes only if both boards agree
DEPLOYED	Both AND gate inputs HIGH simultaneously	HW AND gate output HIGH	Physical deployment signal driven; DWI remains OFF if no fault; boards continue monitoring
FAULT / INHIBIT	Any fault flag active	system_fault=1	D8=LOW, D9=LOW forced immediately (SWR-14); D10=HIGH (DWI); UART timeout or POST fail triggers this
SAFE STATE	Unrecoverable fault / WDT expiry	WDT reset or persistent fault_x=1	MCU hardware reset; GPIO defaults LOW before re-init; all deployment outputs inhibited

DEPLOYED state requires both boards to drive their respective D8 HIGH simultaneously. The AND gate enforces this physically. A single board cannot cause deployment regardless of its software state.

9.2 State Transition Diagram

Transitions evaluated every loop cycle:

- IDLE → MONITORING: POST complete, no fault detected
- MONITORING → CRASH DETECTED: crashLocal = true on current board AND system_fault = false
- CRASH DETECTED → DEPLOYED: both boards drive D8 HIGH simultaneously → AND gate output HIGH
- CRASH DETECTED → MONITORING: crash subsides (crashLocal returns false)
- MONITORING / CRASH DETECTED → FAULT / INHIBIT: any fault_x flag becomes true
- FAULT / INHIBIT → MONITORING: fault clears (only fault_dueX can recover; POST faults are permanent until reset)
- Any state → SAFE STATE: WDT expiry; MCU reset; GPIO defaults LOW

9.3 Deployment Decision Logic (Pseudo-Code)

```
// Both boards execute this every loop iteration
FUNCTION deploymentLogic(crashLocal, babySeat, system_fault):
  // Safe-state enforcement - SWR-14
  IF system_fault THEN
    D8 = LOW; D9 = LOW
    RETURN
  END IF

  // Airbag 1 - driver (SWR-12)
  D8 = crashLocal AND NOT system_fault

  // Airbag 2 - passenger with suppression (SWR-13)
  D9 = crashLocal AND NOT babySeat AND NOT system_fault

  // AND gate enforces: BOTH D8 lines must be HIGH for deployment
  // Software on one board alone cannot cause deployment
```

9.4 Crash Detection Logic (Pseudo-Code)

```
// SWR-05: Convert raw ADC to physical value
FUNCTION toPhys(raw, maxVal):
    RETURN (raw / 4095.0) * maxVal

// SWR-06 + SWR-10: Threshold comparison and OR-fusion
FUNCTION crashDetect(rawA0, rawA1, rawA2):
    accel_g    = toPhys(rawA0, 100.0)    // ACCEL_MAX
    gyro_dps   = toPhys(rawA1, 360.0)    // GYRO_MAX
    press_kPa  = toPhys(rawA2, 500.0)    // PRESS_MAX

    accelEx = (accel_g    >= 30.0)        // THRESH_ACCEL
    gyroEx  = (gyro_dps   >= 120.0)       // THRESH_GYRO
    pressEx = (press_kPa  >= 200.0)       // THRESH_PRESS

    RETURN accelEx OR gyroEx OR pressEx // OR-fusion: SWR-10
```

9.5 Baby Seat Debounce (Pseudo-Code)

```
// SWR-07/08/09 - D11 active-LOW, ext. 10 kΩ pull-up to 3.3 V
FUNCTION updateBabySeat():
    currentRaw = digitalRead(D11)
    IF currentRaw != lastRawBabySeat THEN
        debounceStart = millis()
        lastRawBabySeat = currentRaw
    END IF
    IF (millis() - debounceStart) >= 10 THEN // DEBOUNCE_MS
        stableBabySeat = currentRaw
    END IF
    // Active-LOW: LOW = baby seat present
    babySeat = (stableBabySeat == LOW)
    // suppress-on-fault: if signal never stabilises, babySeat stays true
```

9.6 UART Heartbeat and Timeout (Pseudo-Code)

```
// SWR-17: Non-blocking HB TX every 500 ms
IF (millis() - last_hb_tx) >= 500 THEN
    Serial1.println("ECU1:HB") // or ECU2:HB
    last_hb_tx = millis()
END IF

// SWR-18: HB RX and 2000 ms timeout
IF Serial1.available() THEN
    msg = Serial1.readStringUntil('\n')
    IF msg.contains("ECU2:HB") THEN // Due 1 perspective
        last_hb_rx = millis()
        fault_due2 = false
    END IF
END IF
IF (millis() - last_hb_rx) > 2000 THEN // COMM_TIMEOUT
    fault_due2 = true
```

```

END IF

// SWR-19: Due 2 status TX every 100 ms
IF (millis() - last_status_tx) >= 100 THEN
  Serial1.println("ECU2," + crashLocal + "," + babySeat + "," + system_fault)
  last_status_tx = millis()
END IF

```

9.7 CAN Frame Transmission (Pseudo-Code)

```

// SWR-22: CRC-16 computation
FUNCTION crc16(data[6]):
  crc = 0xFFFF // seed
  FOR each byte b in data[0..5]:
    crc = crc XOR (b << 8)
    FOR 8 iterations:
      IF crc & 0x8000: crc = (crc << 1) XOR 0x1021
      ELSE:          crc = crc << 1
  RETURN crc & 0xFFFF

// SWR-20/21/23: CAN TX every 100 ms — Due 1 only
IF (millis() - last_can_tx) >= 100 THEN
  CAN_FRAME frame
  frame.id = 0x100 | frame.length = 8
  frame.data.bytes[0] = flags_byte // crash+baby+fault bits
  frame.data.bytes[1] = scale(rawA0) // Due1 accel 0-255
  frame.data.bytes[2] = scale(rawA1) // Due1 gyro 0-255
  frame.data.bytes[3] = due2_status // packed Due2 state
  frame.data.bytes[4] = 0x00 // reserved
  frame.data.bytes[5] = seq_counter++ // rolling 0-255
  crc = crc16(frame.data.bytes[0..5])
  frame.data.bytes[6] = (crc >> 8) & 0xFF // CRC MSB
  frame.data.bytes[7] = crc & 0xFF // CRC LSB
  Can0.sendFrame(frame)
  last_can_tx = millis()
END IF

```

10. SWR-to-Architecture Traceability Matrix

Forward traceability from every software requirement in SRS-AIRBAG-SW-001 Rev 1.0 to the software design element defined in this document:

SWR	Description	FSR	ASIL	Software Design Element
SWR-01	POST execution at boot	FSR-17	ASIL C	MOD-01/02/03 — cpu_test, ram_test, adc_test
SWR-02	No output before POST complete	FSR-17	ASIL D	setup() sequence — POST before loop entry
SWR-03	ADC 12-bit resolution	FSR-01	ASIL D	setup() — analogReadResolution(12)
SWR-04	Sensor read every loop	FSR-01/02	ASIL D	MOD-08 — analogRead A0/A1/A2 every loop

SWR-05	Physical conversion formula	FSR-01	ASIL D	MOD-07 — toPhys()
SWR-06	Threshold comparison	FSR-03/04	ASIL D	MOD-09 — accelEx/gyroEx/pressEx flags
SWR-07	Baby seat INPUT mode	FSR-09	ASIL D	setup() — pinMode(D11, INPUT); ext. 10 kΩ pull-up
SWR-08	10 ms debounce filter	FSR-11	ASIL D	MOD-06 — updateBabySeat()
SWR-09	Suppress-on-fault default	FSR-10	ASIL D	MOD-06 — fault branch: babySeat=true
SWR-10	Sensor OR fusion	FSR-03	ASIL D	MOD-10 — crashLocal = accelEx gyroEx pressEx
SWR-11	No blocking delay in loop	FSR-02	ASIL D	Architecture — millis() timers only
SWR-12	D8 AND gate drive logic	FSR-03/08	ASIL D	MOD-11 — driveAirbag1 (SWR-12 conditions)
SWR-13	D9 passenger suppression	FSR-09/10	ASIL D	MOD-11 — driveAirbag2 + babySeat gate
SWR-14	Fault inhibit both outputs	FSR-20	ASIL D	MOD-13 — enterSafeState: D8/D9 LOW on system_fault
SWR-15	WDT configuration	FSR-22	ASIL D	MOD-04 — watchdog_setup(); WDV(256×10) ≈ 10 ms
SWR-16	WDT kick every loop	FSR-22	ASIL D	MOD-05 — watchdog_feed() — first call in loop()
SWR-17	UART heartbeat TX 500 ms	FSR-07	ASIL D	MOD-14 — Serial1 TX, non-blocking millis()
SWR-18	UART timeout fault 2000 ms	FSR-07/15	ASIL D	MOD-15/16 — fault_due1/2 on timeout
SWR-19	Due 2 status TX / parseUART	FSR-07	ASIL D	MOD-17 — 'ECU2,<c>,,<f>' every 100 ms
SWR-20	CAN0 init 500 kbps	FSR-13	ASIL B	MOD-19 — Can0.begin(CAN_BPS_500K, 255)
SWR-21	CAN 8-byte frame structure	FSR-13/16	ASIL B	MOD-19 — canSend() frame layout
SWR-22	CRC-16 on CAN frame	FSR-13	ASIL B	MOD-18 — crc16(poly=0x1021, seed=0xFFFF)
SWR-23	CAN TX every 100 ms	FSR-16	ASIL B	MOD-19 — non-blocking millis() timer
SWR-24	DWI on local fault	FSR-19	ASIL A	MOD-12 — D10 HIGH within one loop cycle
SWR-25	DWI on remote fault	FSR-19	ASIL A	MOD-12 — due2_fault → D10 HIGH

11. Normative References

- SRS-AIRBAG-SW-001 Rev 1.0 - Software Requirements Specification (primary input to this document)
- FSRs-AIRBAG-ECU-001 Rev 2.1 - Functional Safety Requirements Specification
- SVVP-AIRBAG-ECU-001 - Software Verification & Validation Plan (test methods, coverage, validation)
- ISO 26262:2018 Part 6 - Road Vehicles - Functional Safety: Software Level
- IEC 61508:2010 Part 3 - Software Requirements (§7.4 Architecture & Design; Annex A, Annex B)
- ISO 11898 - Road Vehicles: Controller Area Network (CAN)
- Atmel SAM3X8E Datasheet - ARM Cortex-M3 (WDT, CAN0 peripheral registers)
- due_can Library - Arduino Due CAN bus library (proven tool, C4.4)
- Lecture SS26 Functional Safety (Fochtman) - IEC 61508-3 design pattern guidance

12. Referenced and Attached Documents

The following external artifacts are integral components of this Software Architecture and Design Specification. They provide the complete visual layout and low-level code implementation details required for full system verification:

Document ID	Document Title	Format	Description
AD-01	2oo2 Airbag ECU - Detailed Software Flowchart	.pdf	Complete, high-resolution visual layout of the deterministic control loop, sensor fusion logic, and multi-fault safety routing.
AD-01-CODE	2oo2 Airbag ECU - Dual-Channel Arduino Source Code	.ino	Complete, modularized, and statically-allocated source code implementing the 2oo2 redundancy logic within the Arduino environment.

13. Abbreviations and Glossary

Abbreviation	Definition
ADC	Analog-to-Digital Converter
ASIL	Automotive Safety Integrity Level (ISO 26262)
ECU	Electronic Control Unit
FSRS	Functional Safety Requirements Specification
FSR	Functional Safety Requirement
GPIO	General Purpose Input/Output
HARA	Hazard Analysis and Risk Assessment
POST	Power-On Self-Test
SADS	Software Architecture and Design Specification
SIL	Safety Integrity Level (IEC 61508)

SM	Safety Mechanism
SPFM	Single-Point Fault Metric
UART	Universal Asynchronous Receiver/Transmitter
WDT	Watchdog Timer
2oo2	Two-out-of-two voting architecture

14. Approval and Sign-Off

Role	Name	Signature	Date
Software Safety Engineer			
Hardware Safety Engineer			
Functional Safety Engineer			
V&V Lead			
Project Supervisor			